

Android Memory Management — Exercise Set

Exercise 1: Heap limit, ART behavior, OOM

Consider below the part of an app that intentionally exceeds heap memory to observe a crash.

```
public class MemoryHogActivity extends AppCompatActivity {  
    List<byte[]> memoryList = new ArrayList<>();  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        try {  
            while (true) {  
                // Allocate 1MB chunks  
                memoryList.add(new byte[1024 * 1024]);  
            }  
        } catch (OutOfMemoryError e) {  
            Log.e("OOM_TEST", "Out of memory reached!");  
        }  
    }  
}
```

1. What will happen when this code runs?
2. Why does the app crash even though the device has free memory?
3. How does ART enforce heap limits?
4. Name the exception thrown.

Exercise 2: Garbage Collection Pressure (Young Gen): Generational GC, frequent allocations → GC pauses

In the following part of code, we simulate GC pressure using excessive object creation in a loop.

```
public void runGcStress() {  
    for (int i = 0; i < 100000; i++) {  
        String temp = new String("GC_TEST_" + i);  
    }  
}
```

We run this on the UI thread vs a background thread.

1. Why does this code trigger frequent garbage collection?
2. Which GC generation is mainly involved?
3. Why do most objects die in the young generation?
4. What is the impact if executed on the UI thread?
5. What is the impact if executed on a background thread?

Exercise 3: Fixing GC Issues (Optimization), Memory pollution, object reuse

Reconsider the code above with the use of StringBuilder:

```
StringBuilder builder = new StringBuilder();

for (int i = 0; i < 100000; i++) {

    builder.setLength(0); // Resets the logical length to 0, does NOT delete or reallocate the buffer, So the
                           // memory is reused instead of creating a new object

    builder.append("GC_TEST_").append(i);

}
```

1. Explain why the above code reduce GC pressure using StringBuilder.
2. Why does the solution reduce GC frequency?
3. What is “memory pollution”?

Exercise 4: Android Process ranking (priority)

Assume each app runs in a separate process. You are given the following apps:

App	Action
A	• Activity is visible on screen (UI) → OnResume() • No service
B	• Activity is visible → onStart() • Uses startService() to play music in background
C	• Activity is not visible → onStop() • Uses bindService() to connect to a running service • No startService()
D	• Activity is not visible → onStop() • Uses startService() for file download • No UI
E	• Activity is not visible → onStop() • No service running • Kept in memory for fast restart

1. Identify process type (rank) of each app
2. Rank them from highest → lowest priority

Exercise 5: Android Process State Timeline

We track 3 apps (A, B, C) over time. Assume each app runs in a separate process where apps *change states over time* and we want to track their state over time. You are given the followings:

App	Action
A (media App)	• Starts with Activity (onResume()) • Activity onPause() and Starts music using startService() • Later user exits Activity (the activity's onStop()) and Service continues • Finally service stops
B (Messaging App)	• Starts Activity (onStart()) • Activity onResume() and Binds to a background service (bindService()) • Activity is closed (the activity's onStop()) and Service remains running independently • Later service stops
C (Simple App)	• Starts Activity (onStart())

	• Activity onResume() then, • User immediately leaves app (the activity's onStop()) • No service used
--	---

We divide the timeline into 4 stages:

T1 → App launched

T2 → User interaction changes

T3 → UI removed / services running

T4 → System cleanup

For **each time stage (T1, T2, T3, T4)**, Identify process state (components) and process type of each app

KEY INSIGHTS – Examples

1. Started Service keeps process alive

App A stays Service process at T3

2. Bound Service does NOT elevate rank

App B drops to Background once Activity is gone

3. No service → immediate degradation

App C quickly becomes Background → Cached

4. Everything eventually becomes Cached

When no active components remain

Exercise 6: Dynamic Process ranking

Assume each app runs in a separate process where apps *change states over time* and we want to track their state over time (T1 → T2 → T3 → T4). You are given the following apps:

App	Action
A	• Starts with a visible Activity (onResume()) • Later user leaves app (onStop()) • No service
B	• Starts with visible Activity (onResume ()) • Starts music using startForeground() • User leaves the Activity (onStop ()), but music continues • Later music stops
C	• Starts with visible Activity (onResume ()) • Binds to a service (bindService()) and activity becomes partially hidden (onPause()) • Activity is fully closed (onStop ()) and Service continues running independently • Later service stops
D	• Starts with no UI • Runs a startService() to download task • Still downloading • Later service stops
E	• Activity starts and becomes visible → onStart() → onResume() • User quickly leaves the app: ◦ Activity goes to onPause() (briefly visible)

	○ Then to onStop() (no longer visible) • No service is started at any point
--	--

For **each time stage (T1, T2, T3, T4)**:

1. Identify process type of each app
2. Rank them from highest → lowest priority

Exercise 7: Process ranking (priority) + Low Memory Killer

We have 5 apps running in the system, each with a memory footprint.

App	Memory usage (MB)	Action
A	180	Foreground Activity
B	220	Foreground Activity + Started Service (music)
C	250	Foreground Activity + Started Service + Cached Activity
D	120	Background Activity (not visible)
E	80	Cached process (no active components)

The system now receives a new app request (App F) that needs memory, but RAM is full (→ LMK must kill one process).

New Event		
App F wants to launch	Needs 200 MB RAM	System has no free memory

1. Give the order of process priority (by ranking).
2. LMK must decide: Which process(es) should be killed?

Exercise 8: LMK + OOM Scores + Dirty Pages

We have 5 apps in memory. Each process has OOM priority, Memory usage, dirty pages, and cost of killing (based on dirty pages).

App	Component	Memory (MB)	Dirty pages (MB)	OOM adj score	Dirty cost
A	Foreground Activity	200	20		
B	Foreground Activity + Started Service	240	80		
C	Background Service only (not bound, music playback, no UI)	180	10		
D	Background Activity (not visible)	150	90		
E	Cached process (no components active)	90	5		

Recall : A dirty page = memory page that has been modified but NOT written back to storage.

Why it matters:

- Clean page → can be dropped instantly
- Dirty page → must be written to disk before reclaim → slower + more expensive to kill

To simplify we will scale the cost on a scale from 0 – 100 (MB) and categorize into 1 of 5 categories (Very Low, Low, Moderate, High, Very High).

The OOM ADJ SCALE (simplified) is given below:

-17 → Foreground app (very safe)

0 → Visible / perceptible app

6 → Service process

12 → Background process

15 → Cached process (first to kill)

LMK prefers killing: LOW priority + LOW dirty cost first.

The system now receives a new app request (App F) that needs memory, but RAM is full.

New Event		
App F wants to launch	Needs 250 MB RAM	System has no free memory

1. Specify the OOM_ADJ score and dirty cost for each App.
2. LMK must decide by taking into consideration the dirty cost: Which App(s) should be killed?
3. Consider now the decisions which are based on a computed score, not just a rule list. This is very close to how Android (via Linux OOM killer / modern LMKD) actually behaves conceptually. Each process gets a final “Kill Priority Score”: Lower score = less likely to be killed. We compute (simplified approach):

Final Score = OOM_ADJ + Memory_Weight + Dirty_Page_Penalty

Where:

Memory_Weight = RAM_MB / 50

Dirty_Page_Penalty = Dirty_MB / 10

Give the score per each App and specify App(s) to be killed to free memory.

Exercise 9: onTrimMemory Handling - Low memory callbacks before LMK

Consider the Code

```
@Override
public void onTrimMemory(int level) {
    super.onTrimMemory(level);

    if (level >= ComponentCallbacks2.TRIM_MEMORY_BACKGROUND) { // Background → safe caches only
        Log.d("MEMORY", "App in background, clearing non-critical cache");
    }
}
```

```

        // Clear in-memory caches (keep disk cache if needed)

        ImageCache.getInstance().clearMemoryCache();

        DataCache.getInstance().trimToSize(0);
    }

    if (level >= ComponentCallbacks2.TRIM_MEMORY_RUNNING_LOW) { // Low → partial eviction

        Log.d("MEMORY", "System running low on memory, trimming aggressively");

        // More aggressive cleanup

        ImageCache.getInstance().evictHalf();

        ObjectPool.getInstance().clearUnused();

    }

    if (level >= ComponentCallbacks2.TRIM_MEMORY_RUNNING_CRITICAL) { // Critical → aggressive
cleanup
        Log.d("MEMORY", "Critical memory pressure, clearing almost everything");

        ImageCache.getInstance().clearAll();

        DataCache.getInstance().clearAll();

        System.gc(); // optional hint, not relied upon

    }

```

Questions

1. What is the purpose of `onTrimMemory()` and When is it called?
2. Explain the cache clearing logic on the above code.
3. How does this delay LMK?
4. Why is this preferred over waiting for GC?

Exercise 10: Memory Leaks

Case 1: Consider the codes below:

<pre> public class LeakyActivity1 extends AppCompatActivity { static Context context; @Override protected void onCreate (Bundle savedInstanceState) { super.onCreate(savedInstanceState); context = this; // Leak! } } </pre>	<pre> public class LeakyActivity2 extends AppCompatActivity { static Context context; @Override protected void onCreate (Bundle savedInstanceState) { super.onCreate(savedInstanceState); context = getApplicationContext(); // Fix } } </pre>
--	--

Questions

1. Why the code in LeakyActivity1 yields to a memory leak and why can't garbage collection fix it?
2. What about the code in LeakyActivity2?

Case 2: Consider the codes below:

<pre> public class LeakyActivity extends AppCompatActivity { private TextView textView; @Override protected void onCreate(Bundle savedInstanceState) { super.onCreate(savedInstanceState); textView = new TextView(this); textView.setText("Starting task..."); setContentView(textView); startTask(); } public void startTask() { new Thread(new Runnable() { @Override public void run() { /* This anonymous inner class holds an ** implicit reference */ to LeakyActivity (outer class) doHeavyWork(); // After work, try to update UI runOnUiThread(new Runnable() { @Override public void run() { textView.setText("Task finished"); } }); } }).start(); } private void doHeavyWork() { try { // Simulate long running task (5 minutes) Thread.sleep(5 * 60 * 1000); } catch (InterruptedException e) { e.printStackTrace(); } } } </pre>	<pre> public class SafeActivity extends AppCompatActivity { private TextView textView; @Override protected void onCreate(Bundle savedInstanceState) { super.onCreate(savedInstanceState); textView = new TextView(this); textView.setText("Starting task..."); setContentView(textView); startTask(); } public void startTask() { new Thread(new TaskRunnable(this)).start(); } // Static inner class → NO implicit reference to // Activity private static class TaskRunnable implements Runnable { private WeakReference<SafeActivity> activityRef; TaskRunnable(SafeActivity activity) { this.activityRef = new WeakReference<>(activity); } @Override public void run() { doHeavyWork(); // Safely retrieve Activity SafeActivity activity = activityRef.get(); if (activity != null && !activity.isFinishing()) { activity.runOnUiThread(new Runnable() { @Override public void run() { // Double-check inside UI thread SafeActivity act = activityRef.get(); if (act != null && !act.isFinishing()) { act.textView.setText("Task finished safely"); } } }); } } } } </pre>
---	---

	<pre> } } }); } private void doHeavyWork() { try { // Simulate long running task (5 minutes) Thread.sleep(5 * 60 * 1000); } catch (InterruptedException e) { e.printStackTrace(); } } } } </pre>
--	---

Questions

1. Why the code in `LeakyActivity` yields to a memory leak and why can't garbage collection fix it?
2. What about the code in `SafeActivity`?

Exercise 11: Measuring Memory (PSS vs RSS vs USS)

Consider the code

```

    ActivityManager am = (ActivityManager) getSystemService(ACTIVITY_SERVICE);

    int pid = android.os.Process.myPid();

    Debug.MemoryInfo[] memInfo = am.getProcessMemoryInfo(new int[]{pid});

    Log.d("MEMORY", "PSS: " + memInfo[0].getTotalPss());

```

1. What does the code provide?
2. Define the metrics RSS, PSS and USS?
3. Which metric is most accurate and why?
4. How does shared memory affect measurements?
5. Compute RSS, PSS and USS on the following scenario where two processes A and B are running and each has:
 - 50 MB private memory
 - 100 MB shared memory (same shared library used by both)
6. Compute RSS, PSS and USS on the following scenario where 3 processes: A, B, C are running and each has:
 - 30 MB private memory
 - Two shared libraries:

- Library X = 90 MB (shared by A, B, C → 3 processes)
- Library Y = 60 MB (shared by A, B only → 2 processes)